

Perception for Manipulation

Zackory Erickson



Overview

Geometric grasp selection

Point clouds

Picking antipodal grasps

Object segmentation

Open vocabulary models

The Perception-Action Loop

- Perception focuses on estimating the state of the environment, x_t , to inform robot control or a robot policy $\pi(a_t | x_t)$
- A typical pipeline:
 - Sensor Data (e.g. RGB-D) $\rightarrow$ State Estimation (Pose/Shape) $\rightarrow$ Planning $\rightarrow$ Control.
 - *Trend*: This modular pipeline is increasingly blurring into end-to-end Visuomotor Learning

Challenges Unique to Manipulation

- **Occlusion:** The robot's own arm often blocks the camera view.
- **Clutter:** Objects are stacked, touching, or inside containers.
- **High Precision:** 1cm error in navigation is often fine; 1cm error in grasping a mug handle causes failure.
- **Textureless & Transparent Objects:** Traditional depth sensors fail on glass and shiny metal.
- **Deformability:** Perception must track state changes (e.g., folding cloth), not just rigid poses.

3D Representations for Manipulation

Point Clouds: (Nx3) unordered points)

- Native to depth sensors.
- Hard to process with CNNs (e.g. requires PointNet++ or Point Cloud Transformer).

Depth images

- Easy to process with CNNs, but handles 3D perception implicitly.

Implicit Representations (NeRFs / Gaussian Splatting):

- Continuous functions
- State-of-the-Art for manipulating translucent objects: Enables "novel view synthesis" to see behind occlusions and high-fidelity geometry.

Manipulation datasets



YCB

<https://www.ycbbenchmarks.com/object-set/>



GraspNet-1 Billion

<https://graspnet.net/>

Geometric grasp selection

Can we grasp objects using just the geometry alone?

Open3D: aside

```
python3 -m pip install open3d numpy==1.26.4
```

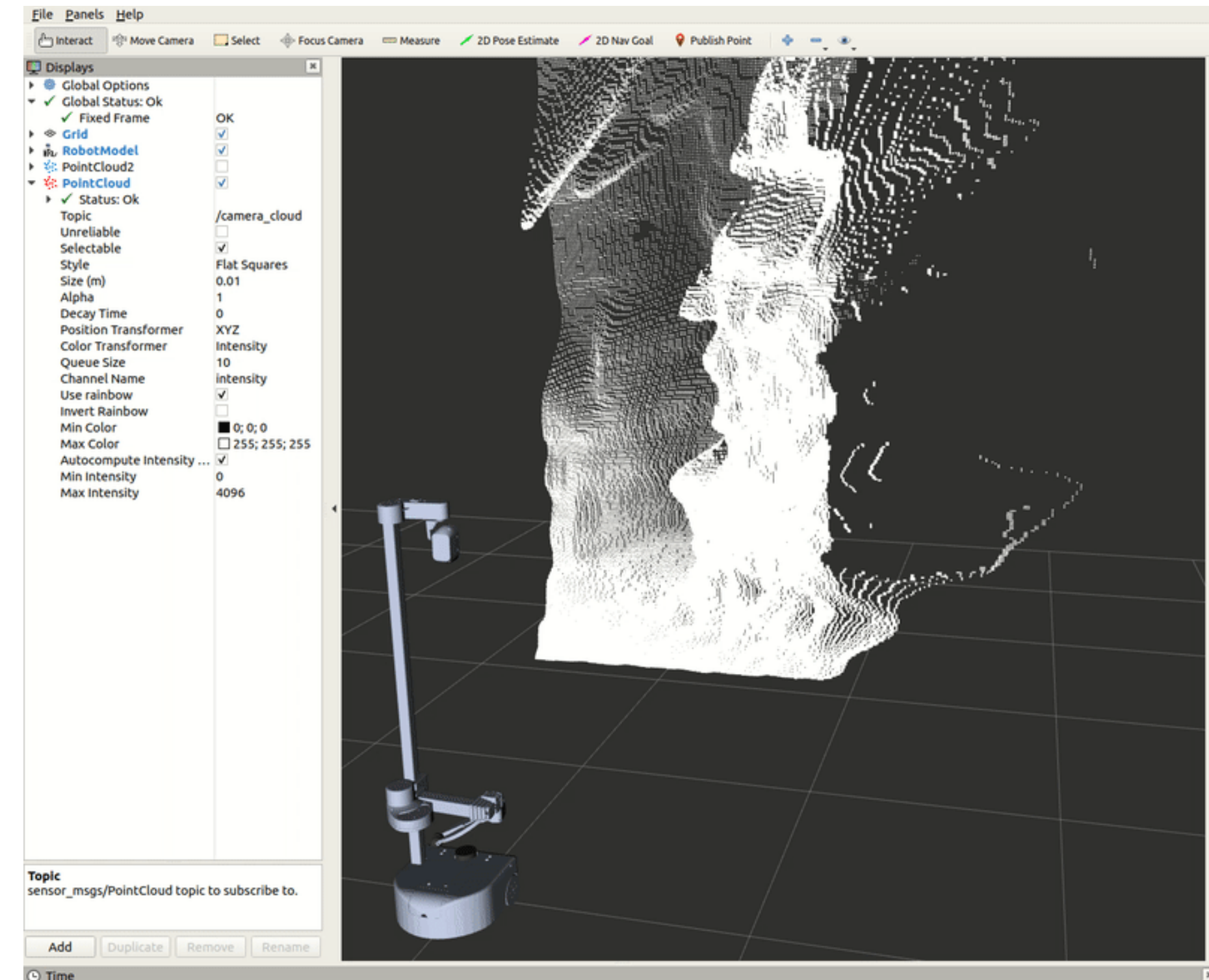
Open3D does not work with recent versions of Numpy.
If you get a seg fault, that may be why.

Capture a point cloud

```
import rclpy, tf2_ros, tf2_sensor_msgs
import numpy as np
from rclpy.node import Node
from sensor_msgs.msg import PointCloud2
import sensor_msgs_py.point_cloud2 as pc2
```

```
class PCSubscriber(Node):
    ... (next slide)
```

```
rclpy.init()
node = PCSubscriber()
rclpy.spin(node)
```



08_pointcloud.py

Capture a point cloud

```
class PCSubscriber(Node):
    def __init__(self):
        super().__init__('pointcloud_subscriber')
        self.tf_buffer = tf2_ros.Buffer()
        self.tf_listener = tf2_ros.TransformListener(self.tf_buffer, self)
        self.sub = self.create_subscription(PointCloud2, '/camera/depth/color/points',
                                             self.callback, 10)

    def callback(self, msg):
        try:
            transform = self.tf_buffer.lookup_transform('base_link',
                                                        'camera_color_optical_frame', rclpy.time.Time())
            pc_transformed = tf2_sensor_msgs.do_transform_cloud(msg, transform)
            pc_numpy = pc2.read_points_numpy(pc_transformed, field_names=('x', 'y', 'z'),
                                             skip_nans=True)
            print('First 3 points:', pc_numpy[:3])
        except tf2_ros.LookupException as e:
            self.get_logger().info(f'Transform not ready: {e}')
```

Point clouds

Normal estimation

`lecture5_pcnormal.py`

`lecture5_pcnormals.py`

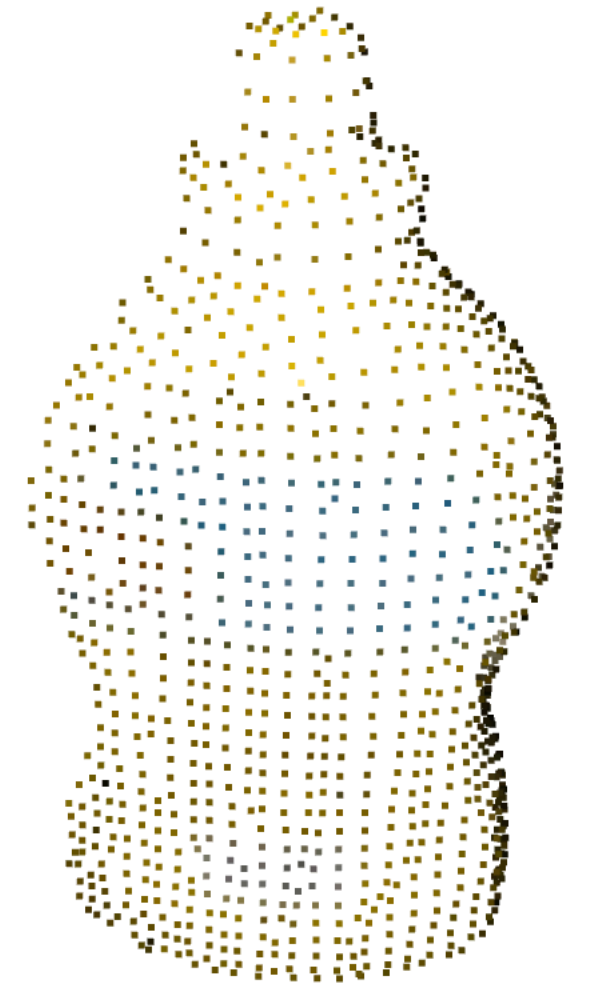
(Press 'n' to see the normals)

This requires installing Manipulation Engine to render a point cloud. <https://github.com/Zackory/mengine/tree/main>

On Stretch, you will replace Manipulation Engine with a real point cloud.

Down sampling

Down sampling



lecture5_pcdownsampling.py

Picking antipodal grasps

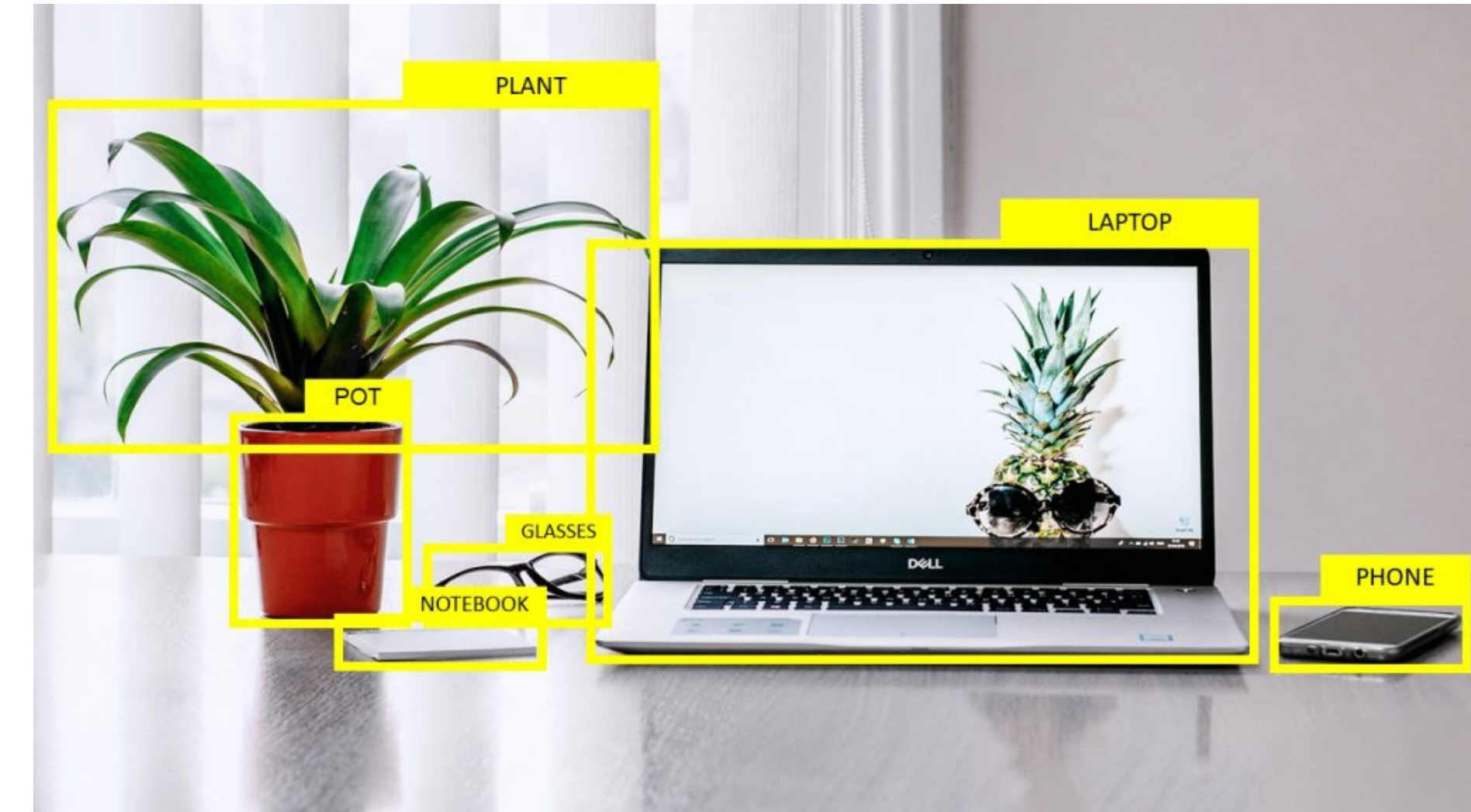
lecture5_antipodal.py

How do we get an object point cloud?

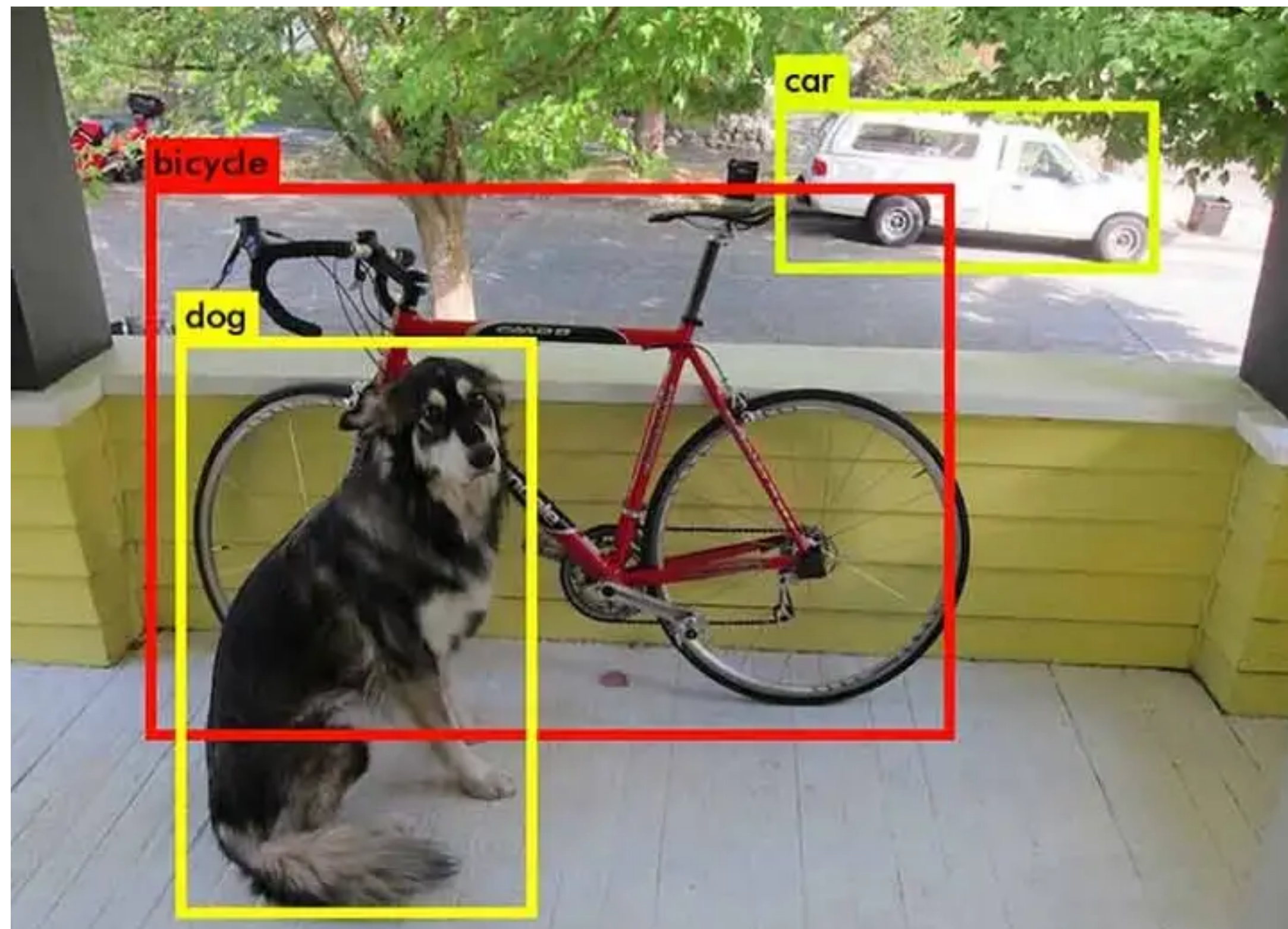
Object Instance Detection

Object Detection (Bounding Boxes):

- Output: Class label + Tuple (x, y, w, h)
- Example architectures: YOLO series, Faster R-CNN.
- Robotic Limitation: A bounding box around a banana **includes the table surface**. Blindly grasping the center of the box can result in collision with the table or a failed grasp on the object's edge.

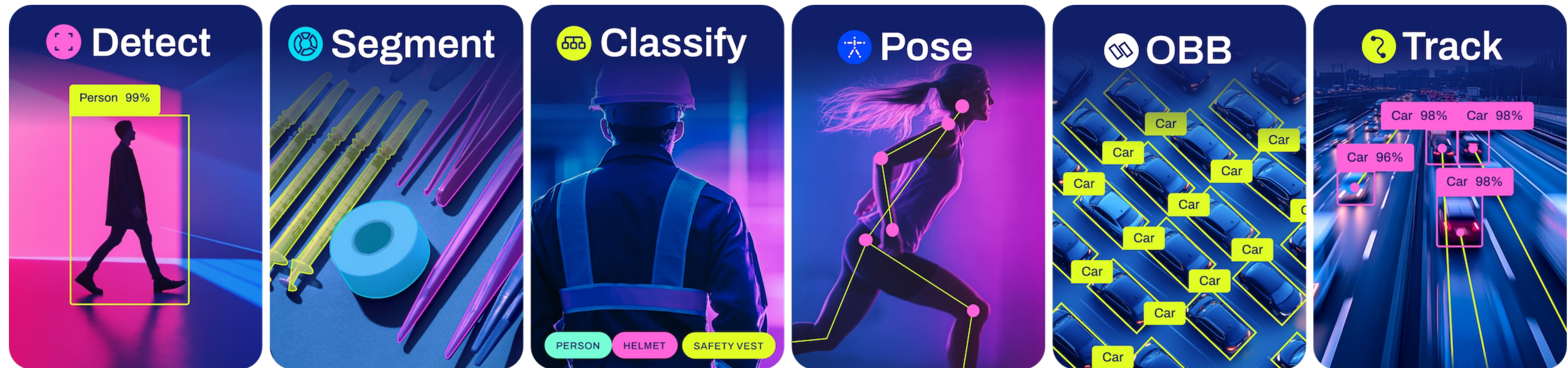


Bounding Boxes for Manipulation



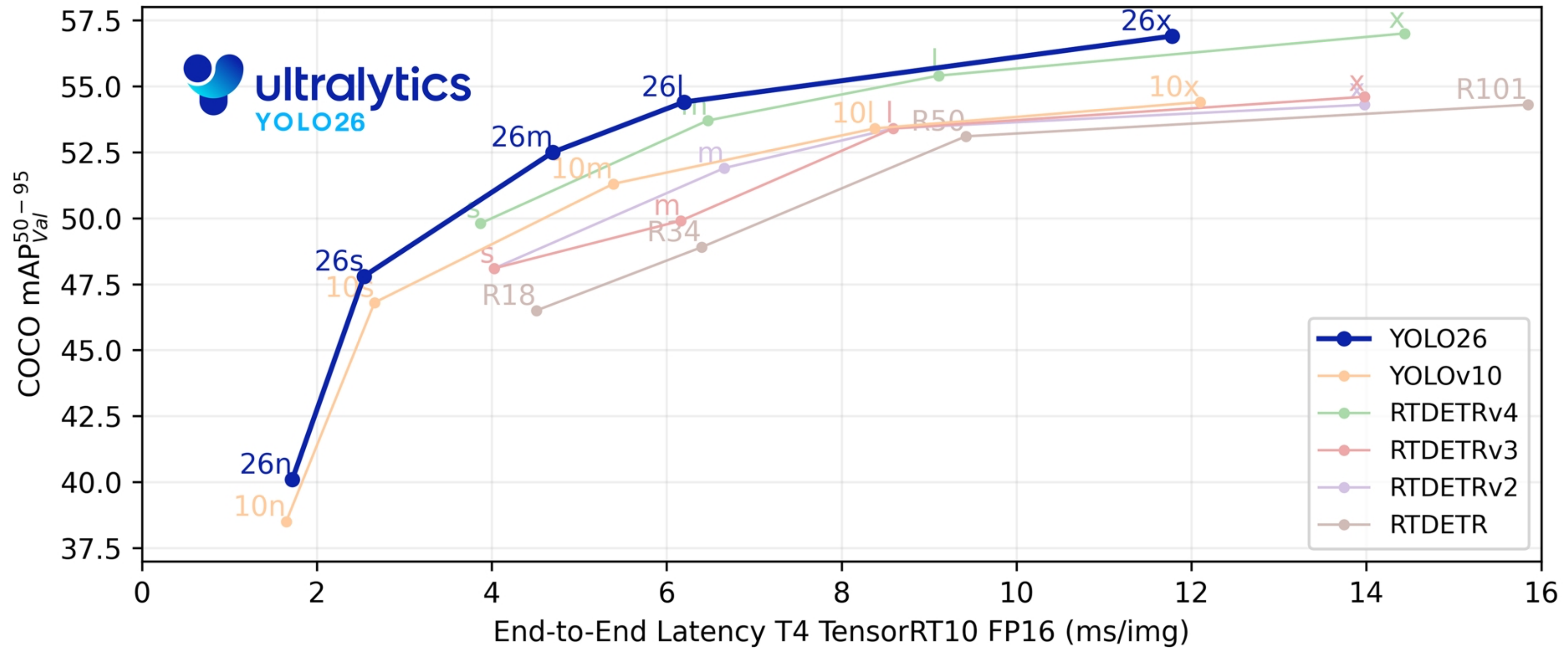
Grasping at the centroid doesn't always work

YOLO26



- Pretrained models: <https://docs.ultralytics.com/models/yolo26/#supported-tasks-and-modes>
- Code example: <https://github.com/ultralytics/ultralytics?tab=readme-ov-file#python>

YOLO26



- YOLO26n runs in real time on Stretch's CPU.
26s may also run in real time.

Object Instance Segmentation

Instance Segmentation (Pixel Masks):

- Output: Binary mask $M \in \{0,1\}^{H \times W}$ for each object instance.
- Example architectures: YOLO, Segment anything (SAM)
- Robotic Advantage: Enables grasp point selection on the actual surface and collision avoidance by defining exact obstacle boundaries.

Object Instance Segmentation

SAM (Segment Anything Model):

- Shift: From closed-set supervision to promptable, zero-shot segmentation.
- Mechanism: A heavy Image Encoder (ViT) runs once. A lightweight Prompt Encoder takes clicks, boxes, or text. A Mask Decoder then produces pixel-level masks.

Real-Time constraints:

- Standard SAM is too heavy (~1-2s on a GPU, even slower on Stretch).
- MobileSAM / FastSAM / EfficientViT-SAM: Distilled versions running at 30+ FPS on edge GPUs (Jetson Orin).

Test out SAM with this image



<https://huggingface.co/spaces/Xenova/segment-anything-web>

Object Instance Segmentation

Video & Temporal Consistency (SAM 2):

- Challenge: Frame-by-frame segmentation flickers, causing control instability.
- Solution: SAM 2 / XMem. Uses a "Memory Bank" to store object features from previous frames.

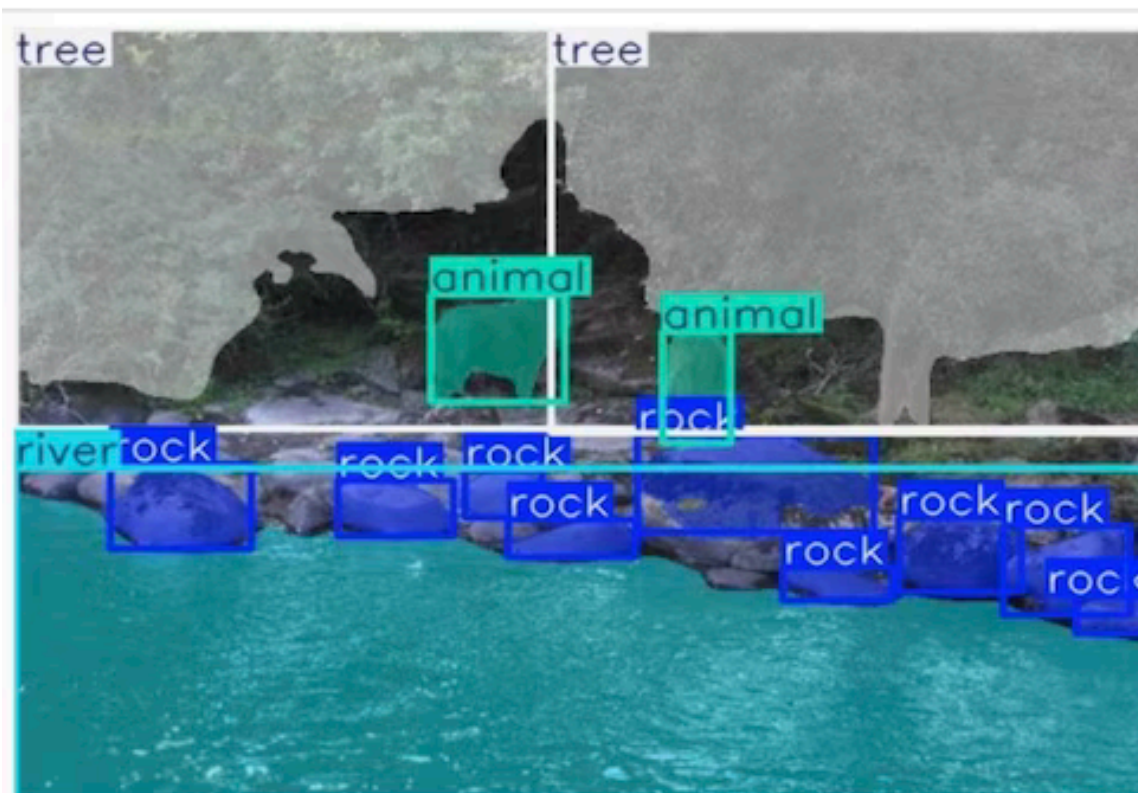
SAM 3: <https://aidemos.meta.com/segment-anything>

- Promptable Concept Segmentation (PCS), enabling the model to detect, segment, and track all instances of a class (e.g., "all cars") using text prompts or image exemplars, rather than just relying on manual clicks.

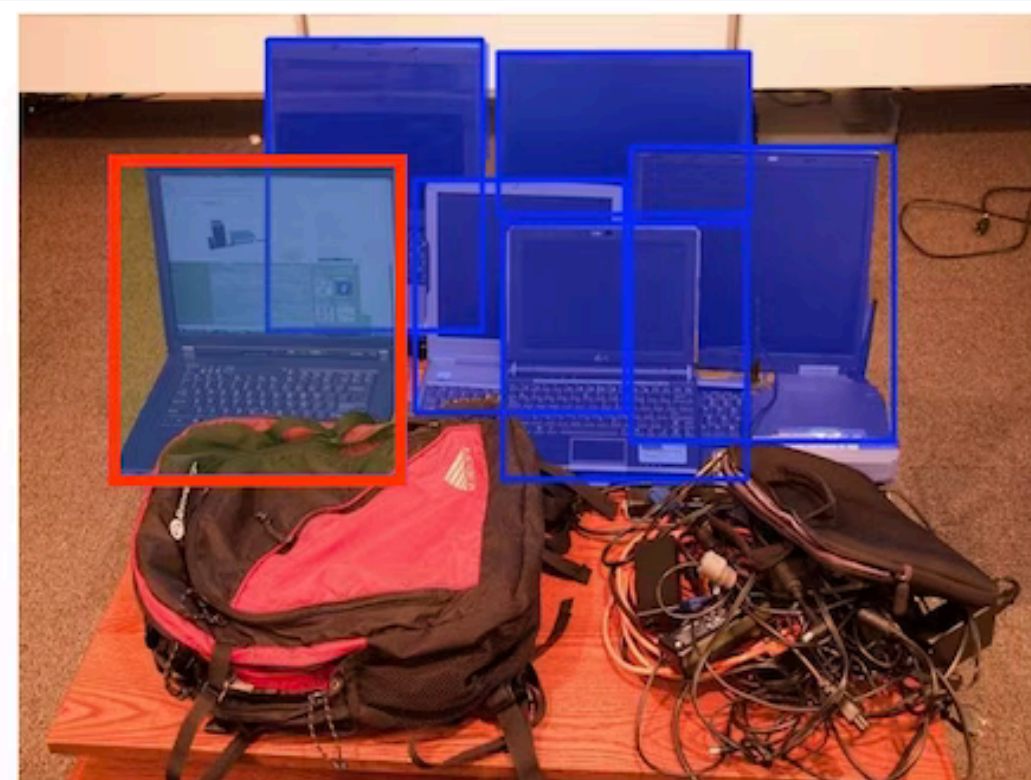
Open Vocabulary Models

- OWL-ViT: https://huggingface.co/docs/transformers/en/model_doc/owlvit
 - OWL-ViT uses Contrastive Language-Image Pre-training (CLIP) as its multi-modal backbone, with a ViT (Transformer) to get visual features and a causal language model to get the text features
 - OWLv2 (https://huggingface.co/docs/transformers/en/model_doc/owlv2) doesn't run in real time on Stretch.
- Grounded SAM 2 (Grounding DINO + SAM 2): <https://github.com/IDEA-Research/Grounded-SAM-2>
- YOLOE-26: <https://docs.ultralytics.com/models/yoloe/>

YOLOE



Text Prompt: {rock, tree, river, animal}



Visual Prompt: box



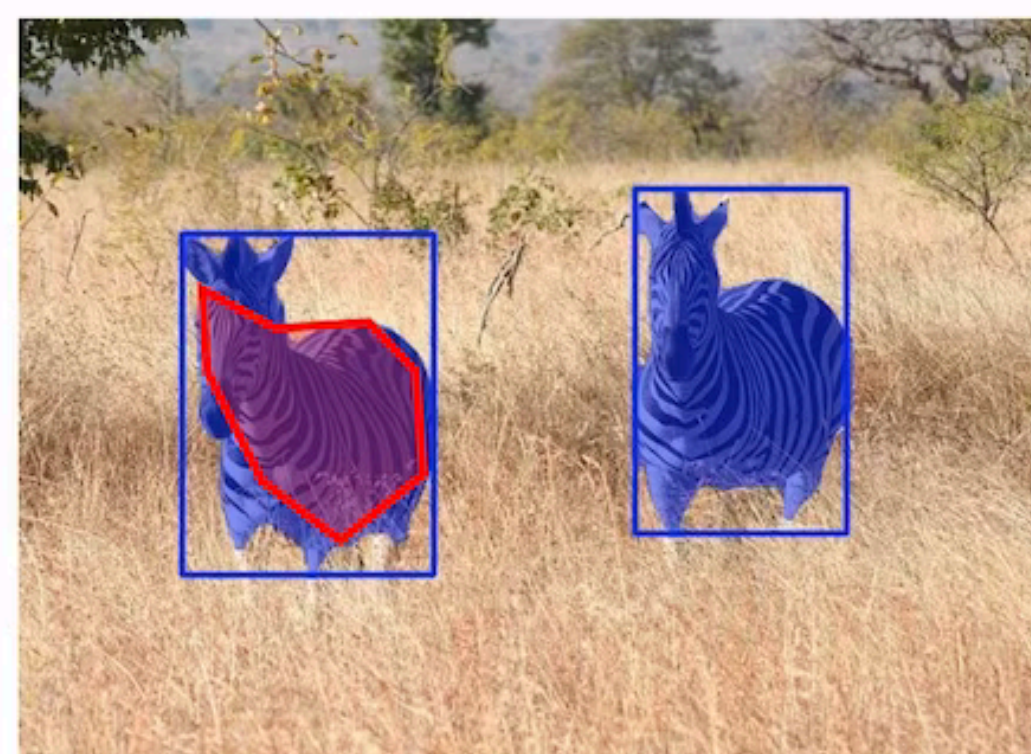
Visual Prompt: cross image



Prompt free (omitting segmentation for clearer visualization)



Text Prompt: {white hat, red hat, white car, sunglasses, mustache, tie}



Visual Prompt: handcrafted shape



Visual Prompt: cross image

Limitations of using geometry only

- No understanding of what an object is.
- Partial views (learning models could infer the back side of objects)
- Ignores friction, mass distribution, grasp stability, dynamics, etc.
- Doesn't work for transparent/translucent objects.